Lab Report: SystemVerilog Simulation Mechanics and Toolchain Fundamentals

Digital Design Verification Lab

May 12, 2026

Abstract

This report details the foundational mechanics of digital hardware simulation using SystemVerilog. It explores the distinctions between hardware compilation and simulation, the event-driven simulation paradigm, and the critical differences in assignment strategies. Furthermore, the report analyzes the open-source compilation toolchain and provides a functional code example demonstrating basic combinational logic and its corresponding test environment.

1 SystemVerilog Assignment Semantics

At the core of hardware modeling is the correct application of assignment operators. Misapplication leads to race conditions in simulation and physical flaws in synthesized silicon.

1.1 Assignment Types

- **Blocking (`=`):** Evaluates and assigns immediately, blocking subsequent execution in the procedural block until complete. It is used exclusively to model combinational logic (`always_comb`).
- **Non-Blocking (`<=`):** Evaluates the right-hand side immediately but defers the assignment to the left-hand side until the end of the simulation time step. It is used exclusively to model sequential logic (`always_ff`).
- **Continuous (`assign`):** Exists outside procedural blocks. It represents a permanent, physical wire connection driven continuously by the right-hand logic.

1.2 Hardware Security Implications

Improper use of blocking assignments in sequential logic creates race conditions. While this manifests as incorrect data in simulation, in physical silicon, poorly structured combinational logic can lead to *glitches*—nanosecond-long voltage spikes before the logic settles. In hardware security, attackers exploit these data-dependent glitches using Power Analysis and Timing Side-Channel Attacks to deduce cryptographic keys. Secure core design mandates logic structures that mitigate these power-leaking glitches.

2 The Simulation Execution Model

Hardware execution is massively concurrent, yet simulation runs sequentially on a host CPU. To resolve this, simulators rely on a stratified event-driven queue, bending time through the concept of the "Delta Cycle."

2.1 Clock Cycles vs. Delta Cycles

- **Clock Cycle (Physical Time):** Represents actual time passing (e.g., nanoseconds). The simulator's internal `$time` variable advances only when physical time elapses.
- **Delta Cycle (Zero Time):** A microscopic, completely hidden execution loop that takes zero physical time. It is used to evaluate massively concurrent logic simultaneously.

2.2 The Stratified Event Queue

Within a single physical timestamp (e.g., exactly $10ns$), the simulator evaluates logic in distinct regions:

1. **Active Region:** Evaluates triggered `assign` statements, `always_comb` blocks, and the right-hand sides of `always_ff` blocks. Blocking assignments (`=`) are updated immediately here. The simulator processes this region until all cascading combinational logic settles.
2. **Non-Blocking Assignment (NBA) Region:** Once the Active Region settles, the simulator processes the deferred non-blocking assignments (`<=`), effectively updating registers and flip-flops.
3. **Feedback Check:** If NBA updates cause combinational signals to toggle, the simulator loops back to the Active Region, initiating a new Delta Cycle at the same physical timestamp.

Waveform viewers (like GTKWave) only record the final, settled state of signals after all Delta Cycles for a given timestamp have concluded.

3 The Open-Source Toolchain

The simulation environment relies on a three-stage pipeline to compile, execute, and analyze hardware definitions.

Tool	Formal Name	Role in Stack
`iverilog`	Icarus Verilog Compiler	Translates SV text into simulation bytecode (`.vvp`).
`vvp`	Virtual Verilog Processor	Execution engine. Calculates logic states and runs system tasks.
`gtkwave`	GTKWave Analyzer	Graphical viewer for Value Change Dump (`.vcd`) files.

Table 1: The Open-Source Simulation Toolchain

4 Module vs. Testbench Architecture

A critical distinction in SystemVerilog is the separation of the Device Under Test (DUT) and the verification environment.

- **Hardware Modules:** Contain port declarations (`input`, `output`). They represent physical, synthesizable logic waiting to be instantiated.
- **Testbench Modules:** Lack port declarations. They represent a self-contained universe that instantiates the DUT, applies internal stimuli via `initial` blocks, and executes system tasks (e.g., `$display`, `$dumpvars`) which are not synthesizable into hardware.

5 Source Code Implementation

The following listing demonstrates a simple combinational AND gate and its corresponding testbench, illustrating module instantiation, signal driving, and compiler directives.

```
module and_gate (
    input logic a,
    input logic b,
    output logic y
);
    always_comb begin : andLogic
        y = a & b;
    end
endmodule

module tb_and_gate;
    // Declare signals to connect to the module
    logic test_a;
    logic test_b;
    logic test_y;

    // Instantiate module for testing (dut)
    and_gate dut (
        .a(test_a),
        .b(test_b),
        .y(test_y)
    );

    // Drive signals
    initial begin
        // For GTKWave - dump file and dump signals
        $dumpfile("dump.vcd");
        $dumpvars(0, tb_and_gate);

        $display("Starting Simulation...");

        // Pass inputs and time wait
        test_a = 0; test_b = 0; #10;
        test_a = 0; test_b = 1; #10;
        test_a = 1; test_b = 0; #10;
        test_a = 1; test_b = 1; #10;

        $display("Ending Simulation...");
        $finish;
    end
endmodule
```

Listing 1: SystemVerilog source for AND Gate and Testbench